SYSTEM ANALYSIS DOCUMENTATION (SAD)

UMHackathon 2026

umhackathon@um.edu.my

© UMHackathon 2026

Email: umhackathon@um.edu.my

1. Introduction:

SyncBite-AI is an AI-enabled inventory management and decision intelligence platform developed to support campus-adjacent small and medium-sized enterprises (SMEs), such as food trucks, beverage stalls, dessert shops, and canteen operators. These businesses often face unpredictable customer demand caused by sudden changes in campus schedules, weather disruptions, examination periods, and academic events. In reality, many small businesses still rely on manual stock recording methods such as spreadsheets, handwritten notes, or personal judgment, which often lead to inefficiencies, inaccurate stock levels, and poor purchasing decisions instead of benefiting from technology. Therefore, our team decided to develop SyncBite-AI as an innovative solution to modernize inventory management, enhance operational efficiency, and support smarter business decision-making.

1.1. Purpose:

The System Analysis Document outlines the technical scope, architectural structure, and key design decisions behind the development of **SyncBite-AI**. It explains how the system components interact to deliver real-time decision intelligence for campus-adjacent SMEs. The document also provides a clear overview of the technologies, workflows, and implementation strategy adopted throughout the project.

The key elements of the system covered in this document are as follows:

1. **Architecture**:

SyncBite-AI adopts a cloud-based client-server architecture consisting of a React frontend hosted on Vercel, a FastAPI backend deployed on Render, and an external GLM service layer for AI reasoning. Each component operates independently while communicating through secure API requests, enabling modularity, maintainability, and future scalability.

2. **Data Flows:**

The system processes merchant inventory inputs, campus context data, and external signals such as weather forecasts through the backend API. Relevant information is structured and transmitted to the GLM engine, which generates strategic recommendations. The validated output is then returned to the frontend dashboard for merchant review and action.

3. **Model Process:**

The model workflow demonstrates the end-to-end operation of SyncBite-AI, beginning with inventory submission, followed by campus intelligence synchronisation, contextual analysis by the GLM, and generation of actionable business strategies with explainable reasoning and predicted financial impact.

4. **Role of Reference:**

This document serves as a technical reference for developers, testers, judges, and future stakeholders by providing a high-level understanding of the system design, operational logic, and implementation feasibility of SyncBite-AI.

1.2. **Background**:

© UMHackathon 2026

Campus-based SMEs frequently depend on student foot traffic as their primary source of income. Traditionally, merchants prepare inventory based on past experience or intuition. However, modern campus environments are highly dynamic. Examination schedules may change, lectures may shift online, and heavy rainfall may significantly reduce customer traffic.

Without access to real-time contextual intelligence, merchants risk producing excess food, wasting perishable inventory, or missing potential sales opportunities.

SyncBite-AI addresses this challenge by combining live campus context, weather signals, and merchant inventory data with artificial intelligence. The platform provides timely and explainable business strategies, allowing merchants to adapt quickly to changing conditions.

1.3. Target Stakeholder

Stakeholders	Roles	Expectations
Food Truck Operators	Sell food near campuses	Better demand forecasting and waste reduction
Canteen Operators	Serve campus communities	Real-time planning insights
Beverage / Dessert Stall Owners	Manage perishable products	Smarter promotions and stock control
SME Owners	Protect revenue and improve efficiency	Clear and actionable AI recommendations
Development Team	Build and maintain system	Scalable and modular architecture
Investors	Evaluate solution feasibility	Innovation, practicality, and growth potential

2. System Architecture & Design

2.1. High Level Architecture

2.1.1. Overview:

Type	Details
System	Responsive Web Application (Desktop & Mobile Browser)

© UMHackathon 2026

Architecture	Cloud-Based Client-Server Architecture with AI Service Layer
Users	Campus SMEs, Food Stall Operators, Food Truck Vendors
Deployment	Frontend on Vercel, Backend on Render
AI Layer	Z.AI GLM API
External Services	Weather API, Campus Calendar Sources

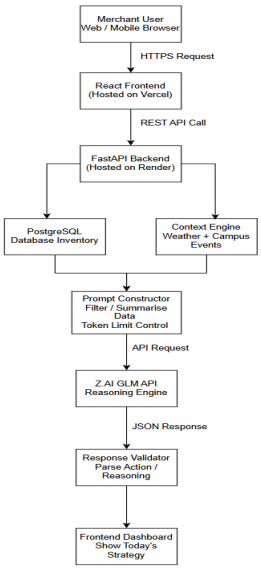
SyncBite-AI is designed as a modern client-server platform where merchants access the system through a responsive web dashboard using desktop or mobile browsers. The frontend communicates with a FastAPI backend through RESTful APIs. The backend manages business logic, contextual data aggregation, prompt construction, and communication with the Z.AI GLM service.

Unlike a monolithic structure, the system separates the presentation layer, backend logic, and AI reasoning layer into independent components. This modular design improves maintainability, scalability, and deployment flexibility.

2.1.2 LLM as Service Layer

Artificial intelligence is implemented as a dedicated reasoning service rather than a generic add-on module. The Z.AI GLM functions as the core decision engine that analyses multiple data sources simultaneously and generates business recommendations.

2.1.3 Dependency Diagram



© UMHackathon 2026

SyncBite-AI integrates GLM as a dedicated AI service layer instead of treating it as a general AI module. When the merchant inputs inventory data through the React dashboard, the frontend sends the data to the FastAPI backend through a REST API request.

The backend then builds a context package by combining the merchant's inventory information with campus calendar data and weather-related signals. This context package is passed into the Prompt Constructor, which formats the information into a structured prompt for the Z.AI GLM API.

The context window includes product quantity, expiry status, cost price, selling price, campus condition, upcoming events, examination periods, and weather warnings. To control token usage, the system filters out low-risk or safe inventory items before sending the prompt. If the input is too large, the backend summarises or chunks the data based on urgency, recency, and business relevance.

After the GLM processes the prompt, it returns a structured response containing the recommended action, reasoning, secondary tip, and predicted financial impact. The FastAPI backend parses and validates this response before forwarding it to the React frontend. The merchant then reviews the recommendation and manually decides whether to execute the strategy.

The following API calls show how the distributed components of SyncBite-AI communicate with each other. Since the frontend, backend, AI service, and external data sources are deployed or accessed separately, the system depends on API communication to transfer data between components.

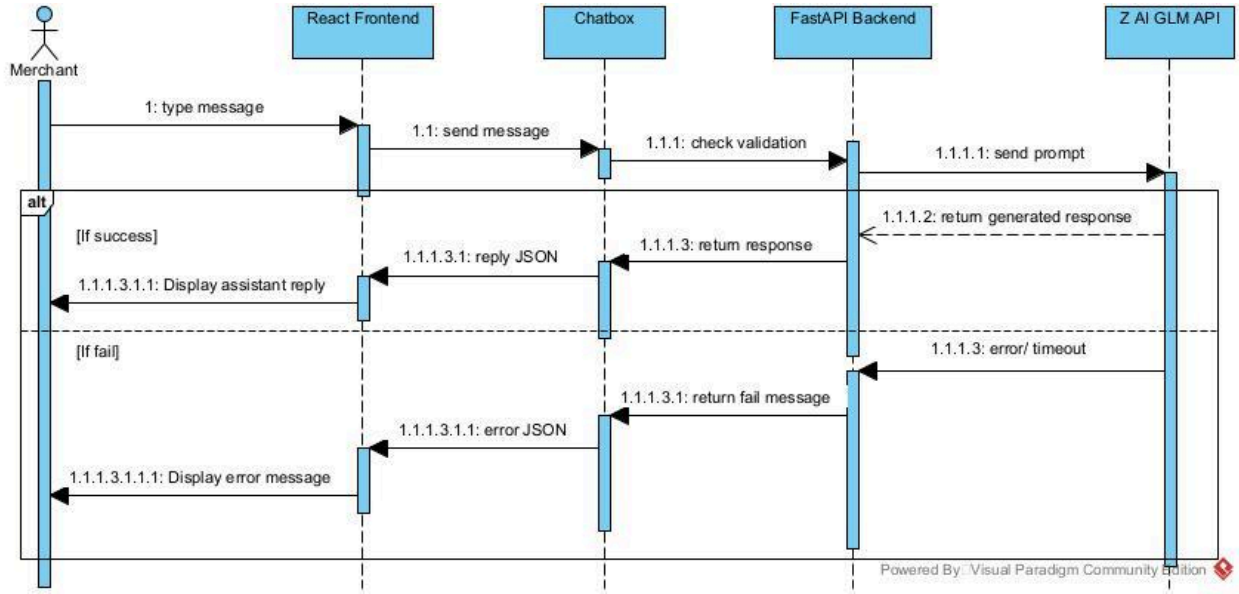
Source	Destination	API / Call Purpose
React Frontend	FastAPI Backend	Submit inventory and merchant request
FastAPI Backend	Weather API	Retrieve weather conditions
FastAPI Backend	Campus Calendar Source	Retrieve campus events and academic schedule
FastAPI Backend	Z.AI GLM API	Send structured prompt for AI strategy generation
Z.AI GLM API	FastAPI Backend	Return AI-generated recommendation
FastAPI Backend	React Frontend	Send parsed strategy output to dashboard

© UMHackathon 2026

SyncBite-AI is deployed across multiple services. The React frontend is hosted on Vercel, while the FastAPI backend is hosted separately on Render. The backend acts as the central coordination layer because it connects the user interface, weather service, campus calendar source, and Z.AI GLM API.

This means the frontend does not communicate directly with the GLM or external APIs. Instead, all requests pass through the backend to ensure better control, security, validation, and response formatting. This distributed design improves flexibility because each component can be updated, scaled, or replaced independently without affecting the whole system.

2.2.2. Sequence Diagram



The sequence diagram shows how the AI chat feature in SyncBite-AI operates from start to finish. First, the merchant types a message on the React Frontend, which is passed to the Chatbox component and sent to the FastAPI Backend. The backend validates the request and forwards the prompt to the Z.AI GLM API for response generation. If successful, the generated reply is returned to the backend, then sent back to the Chatbox and displayed on the frontend for the merchant. If the API fails or times out, the backend returns an error message, which is shown to the user. This process demonstrates the communication between the frontend, backend, and AI service layer.

2.2. Technological Stack

Layer	Technology	Purpose
Frontend	React.js	Responsive and interactive user interface

Styling	Tailwind CSS	Fast UI development
Backend	FastAPI	High-performance REST API framework
Programming Language	Python	Strong API and AI ecosystem
AI Engine	Z AI GLM	Contextual reasoning and insight generation
Hosting (Frontend)	Vercel	Fast static deployment
Hosting (Backend)	Render	Easy cloud backend hosting
Version Control	GitHub	Collaboration and deployment workflow

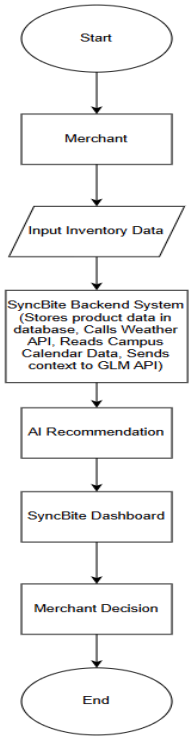
2.3. Key Data Flows

In the current SyncBite-AI MVP, data mainly flows between the React frontend, the FastAPI backend, and the Z.AI GLM API. The frontend manages user interactions such as dashboard viewing, inventory display, strategy pages, and chat input. When the merchant sends a message through the AI Chatbox, the message is passed from the React frontend to the FastAPI backend through a chat request. The backend validates the message, structures it into a prompt, and sends it to the Z.AI GLM API. After the GLM generates a response, the backend returns the reply in JSON format to the frontend, where it is displayed to the merchant.

For the current prototype, inventory and strategy data are mainly handled within the frontend using predefined or local data. In a future production version, these records can be stored in a relational database using structured tables such as merchant, inventory item, chat message and strategy.

2.3.1. Data Flow Diagram (DFD)

© UMHackathon 2026

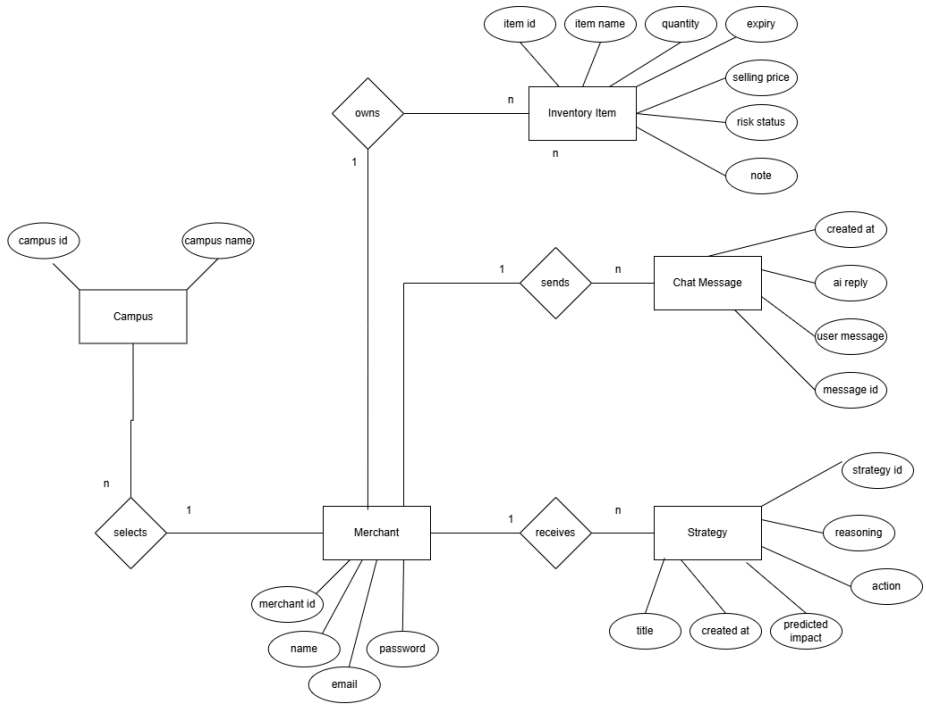


1. Merchant enters inventory information.
2. Data is stored in structured tables.
3. Backend enriches data with weather and campus context.
4. Combined context is analysed by GLM.
5. Output recommendation returned to merchant dashboard.

This avoids black-hole processing because every input has a visible destination and output.

2.3.2. Normalized Database Schema

© UMHackathon 2026



The ERD for SyncBite-AI is centred around the Merchant entity, which stores user account details such as merchant ID, name, email, and password. A merchant can own many Inventory Item records containing stock details such as item name, quantity, expiry, selling price, risk status, and notes. A merchant can also send many Chat Message records through the AI assistant and receive many Strategy recommendations generated by the system. The Campus entity stores campus information, where a merchant can select a campus for contextual business insights. This database design follows Third Normal Form (3NF) by separating data into related entities to reduce redundancy and maintain data integrity.

3. Functional Requirements & Scope

The project focuses on demonstrating a working AI-assisted decision support prototype through a React frontend and FastAPI backend.

3.1. Minimum Viable Product:

3.1.1. Functions

#	Features	Description
1	Dashboard Overview	Displays the main business dashboard with daily performance, risk indicators, and visual summaries.
2	Inventory Page	Shows inventory-related information and stock risk presentation for the merchant.

© UMHackathon 2026

3	Campus Intelligence Page	Displays campus-related context to support business planning.
4	Strategy Page	Presents strategic business recommendations and decision-support content.
5	AI Chat Assistant	Allows merchants to type a message, send it to the FastAPI backend, and receive a response from the Z.AI / ILMU GLM model.

3.2. Non-Functional Requirements (NFRs)

Quality (e.g.)	**Requirements (e.g.)**	**Implementation (e.g)**
Usability	The system should be simple for SME merchants to use.	React frontend provides clear pages such as Dashboard, Inventory, Campus, Strategy, Chat, and Profile.
Performance	The AI chat response should return within a reasonable time.	FastAPI backend sends requests to ilmu-glm-5.1 with timeout handling.
Reliability	The system should not crash when the AI API fails.	Backend returns an error response if the GLM API request fails.
Maintainability	The system should be easy to update.	Frontend pages and backend API are separated into different files and components.
Deployment Readiness	The prototype should be accessible online.	Frontend can be deployed on Vercel, while backend can be deployed separately on Render.
AI Intefration	The system should demonstrate real GLM usage.	Chatbox endpoint connects the backend to the ILMU / Z.AI GLM API.

3.3. Out of Scope / Future Enhancements

The following features are part of the larger product vision but are not fully implemented in the current prototype:

- Real database storage for merchant, inventory, chat history, and strategy logs

© UMHackathon 2026

- Live weather API integration
- Live campus calendar API integration
- POS system integration
- Automatic promotion execution
- Multi-user merchant accounts
- Native mobile application
- Supplier auto-ordering
- Real-time social media sentiment analysis
- IoT stock tracking or smart inventory sensors

4. Monitor, Evaluation, Assumptions & Dependencies

4.1. Technical Evaluation:

4.1.1. Grayscale Rollout & A/B Testing:

For future production deployment, new SyncBite-AI features should be introduced gradually through a grayscale rollout strategy. New updates such as improved AI prompts, dashboard widgets, or strategy layouts can first be released to a small group of users (for example 10%) while system stability and user feedback are monitored. If no critical issues are detected, deployment can be expanded progressively to all users.

A/B testing can also be applied to compare different interface designs or AI recommendation styles. For example, two versions of the Strategy Page may be tested to determine which version achieves better user engagement or recommendation acceptance rates.

4.1.2. Emergency Rollback (ER) & Golden Release:

If a new deployment causes critical failures such as frontend crashes, broken API requests, or invalid AI responses, the system should immediately roll back to the last stable version (Golden Release). This ensures service continuity while the faulty version is investigated and corrected.

4.1.3. Priority Matrix:

Priority	Trigger Condition	Action Taken
P1 Critical	Frontend unavailable or chat endpoint fails completely	Immediate rollback to stable version
P2 High	GLM API response errors exceed acceptable threshold	Activate fallback response and investigate API issue

© UMHackathon 2026

P3 Medium	Slow AI responses or minor UI bugs	Hotfix patch in next release cycle
P4 Low	Cosmetic issues or text corrections	Fix during scheduled maintenance

4.2. Monitoring:

System health should be continuously monitored through the following indicators:

- Frontend uptime and page loading errors
- Backend API availability and response time
- Chat request success or failure rate
- GLM API latency and timeout frequency
- Browser console errors and frontend crashes
- User interaction completion rates

If abnormal spikes in failures or latency are detected, alerts should notify the development team for immediate review.

4.3. Assumptions:

The following assumptions were made during development and deployment:

- Users have stable internet access to use the web application.
- Merchants use modern browsers on desktop or mobile devices.
- Users input meaningful messages when using the AI Chat Assistant.
- The ILMU / Z.AI GLM API remains available during normal operation.
- Hosting platforms such as Vercel and Render remain operational.

4.4. External Dependencies:

Tools	Purpose	Risks
Z.AI / ILMU GLM API	Core AI reasoning engine for chat responses and strategy generation	High – service outage or rate limit affects AI features
Vercel	Hosts React frontend application	Low – deployment or CDN issue may affect frontend access
Render	Hosts FastAPI backend service	Medium – backend downtime affects API communication
GitHub	Source code version control and deployment workflow	Low – temporary access issues may delay updates

© UMHackathon 2026

Web Browser	User access platform	Low – compatibility differences across browsers

5. Project Management & Team Contributions

The duration of the Preliminary round is approximately 10 days considering the two-week sprint.

5.1. **Project Timeline:** Show the timeline of development and justify accordingly [e.g. Day 1-2, Project Planning etc.]

Days	Focus Area	Key Activities
1–2	Planning & Design	Use case modelling, database schema finalisation, UI/UX wireframing.
3–5	Core Development	Frontend UI components, Backend API endpoints (FastAPI), and DB integration.
6–7	Integration & Testing	API connection (Frontend-Backend), bug fixing, and user flow testing.
8	Deployment & Demo Prep	Hosting on Render/Vercel, preparing slide deck, recording demo.
9–10	Final Documentation	Refining technical reports, UML diagrams, and final project sign-off.

5.2. **Team Members Role:** Highlight the role of team members [e.g. Person A – Role Backend Lead – Fast API Microservices & Schema Design]

1. Tan Chin Wen (Backend Lead): Responsible for FastAPI microservices architecture, database schema design, and API integration with the LLM (Z.AI).
2. Chong Wei En Chloe (Frontend Lead): Responsible for React component development, state management (using useState/useEffect), and ensuring a responsive mobile-first UI.
3. Lim Yin Jie (Demo & Strategy Lead): Responsible for the final presentation structure, demonstration walkthrough, and managing the campaign strategy logic.
4. Yap Qiao Yan (Documentation Lead): Responsible for UML diagrams (Class, Sequence, Activity), project requirement documentation, and ensuring all deliverables meet the assessment criteria.

© UMHackathon 2026

5.3. Recommendations: If any recommendation is applicable in terms of scalability, performance and reliability [e.g. using Redis Cache for Menu data]

- **Scalability (Redis Caching):** Implement Redis to cache frequently requested data like the "Daily Menu." This reduces database load and significantly speeds up response times.
- **Performance (Asynchronous Task Queue):** For heavy operations (like sending mass notifications to WhatsApp groups), use Celery with RabbitMQ/Redis. This allows your API to remain responsive while heavy tasks are processed in the background.
- **Reliability (Monitoring & Logging):** Integrate Sentry or Grafana to monitor real-time API performance and capture error logs, ensuring you are alerted to issues before users report them.
- **Database (Connection Pooling):** If moving to a larger database, utilise an ORM-based connection pooler (like SQLAlchemy's built-in pool) to handle high-concurrency requests efficiently.

© UMHackathon 2026